

UNIVERSIDAD AUTÓNOMA DEL BENI
"JOSÉ BALLIVIÁN"
FACULTAD DE INGENIERÍA Y TECNOLOGÍA
INGENIERÍA EN SISTEMAS



***Simulación del Microprocesador Intel 8086 y su
Interfaz con Periféricos mediante Proteus***

Universitario: Joel Leiva Calle

Docente: Ing. José Machicado Terán

Asignatura: Arquitectura de computadoras

Semestre: 5to semestre

Gestión: 2025

Trinidad – Beni -Bolivia

Objetivos del Proyecto

Objetivo General:

Diseñar, implementar y simular un circuito electrónico basado en el microprocesador Intel 8086 y el Controlador de Interfaz Periférica Programable (PPI) 8255, utilizando el entorno de simulación Proteus VSM. Asimismo, programar su funcionalidad principal en lenguaje ensamblador mediante el uso del simulador EMU8086, con el fin de mostrar mensajes predefinidos en un display de 14 segmentos.

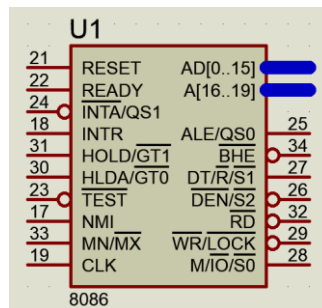
Objetivos Específicos:

- Familiarizarse con la arquitectura del Intel 8086: Comprender el funcionamiento del bus de datos de 16 bits y el bus de direcciones de 20 bits, así como su arquitectura Von Neumann. Analizar la multiplexación de líneas de direcciones y datos y el uso de la señal ALE/QS₀ para su separación mediante un latch externo.
- Dominar la programación en lenguaje ensamblador 8086: Desarrollar habilidades para escribir, depurar y estructurar programas en lenguaje ensamblador, considerando las secciones fundamentales (.model, .stack, .data, .code), el uso de macros, y la creación de subrutinas para mejorar la organización y reutilización del código.
- Implementar la interfaz con el PPI 8255: Configurar correctamente el chip 8255 para operar en modo 0, asignando los puertos A, B y C como salidas, y utilizarlos para enviar datos a los dispositivos conectados, como los displays.
- Simular el circuito completo en Proteus VSM: Diseñar y simular el sistema utilizando el entorno ISIS de Proteus, conectando el microprocesador, el latch 74HC373, el inversor lógico, el PPI 8255, y los displays de 14 segmentos. Evaluar el funcionamiento del circuito en tiempo real junto con la ejecución del código ensamblador.
- Desarrollar una aplicación funcional: Crear un programa en ensamblador que permita visualizar, de forma indefinida, mensajes o palabras predefinidas en los displays, utilizando técnicas de multiplexado y control por software.
- Comprender la interacción hardware–software: Analizar cómo las instrucciones de bajo nivel afectan directamente al estado de los pines y registros del hardware simulado, entendiendo la relación entre el código ensamblador y el comportamiento del circuito electrónico en Proteus.

Introducción

Microprocesador Intel 8086

El Intel 8086 es un microprocesador de 16 bits lanzado por Intel en 1978, que se destacó por su arquitectura Von Neumann y su capacidad para direccionar hasta 1 MB de memoria mediante un bus de direcciones de 20 bits. Este procesador, considerado uno de los precursores de la arquitectura x86, se utilizó ampliamente en sistemas de computadoras personales y dispositivos embebidos.

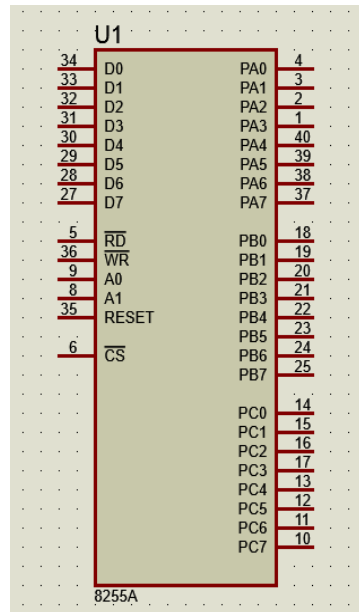


El Intel 8086 tiene varias partes clave que permiten su funcionamiento, entre las que destacan:

- **Bus de Datos (16 bits):** Este bus permite la transferencia de datos entre el procesador y la memoria o los periféricos, trabajando con un ancho de 16 bits, lo que le otorga mayor velocidad de procesamiento frente a los microprocesadores de 8 bits anteriores.
- **Bus de Direcciones (20 bits):** A través de este bus, el 8086 puede direccionar un máximo de 1 MB de memoria (2^{20} direcciones), lo cual era un avance importante en comparación con los microprocesadores anteriores que solo podían manejar 64 KB.
- **Registros:** El 8086 cuenta con varios registros de propósito general, como AX, BX, CX, DX, que permiten almacenar temporalmente datos y direcciones. Además, tiene registros segmentados como CS (Segmento de Código), DS (Segmento de Datos), y SS (Segmento de Pila), que facilitan la gestión de la memoria.
- **Unidad de Control:** Esta unidad es responsable de interpretar las instrucciones de los programas y coordinar las acciones del procesador. El Intel 8086 tiene un conjunto de instrucciones en lenguaje ensamblador que permite realizar operaciones aritméticas, lógicas, de comparación y de manipulación de datos.
- **Pines de Control:** Entre los pines más importantes se encuentran ALE/QS0 (que indica la multiplicación de la dirección en el bus de datos), READY (que sincroniza las señales de entrada/salida), INTR (interrupción), y HOLD (que indica que el bus está siendo utilizado por otro componente), entre otros. Estos pines son fundamentales para la comunicación del procesador con los periféricos y otros dispositivos del sistema.

El 8086 también introduce el concepto de multiplexación de direcciones y datos a través de las líneas ALE/QS0, lo que permite usar las mismas líneas para las direcciones y los datos en momentos diferentes, optimizando el uso del bus.

Intel 8255A (PPI - Programmable Peripheral Interface)



El Intel 8255A es un Controlador de Interfaz Periférica Programable (PPI) diseñado para facilitar la comunicación entre un microprocesador y periféricos a través de puertos paralelos. Este chip es altamente versátil, ya que permite configurar tres puertos de 8 bits (A, B y C) para operar en diferentes modos según las necesidades del sistema. Los puertos A y B pueden configurarse como puertos de entrada o salida, mientras que el puerto C se puede utilizar para controlar señales de control o como entrada/salida en modo especial. El 8255A soporta tres modos de funcionamiento: el Modo 0 (básico) para acceso directo a los puertos de entrada/salida, el Modo 1 (estándar de control) para uso de señales de control adicionales, y el Modo 2 (bidireccional) para comunicación bidireccional en el puerto A.

Este chip se comunica con el microprocesador mediante líneas de dirección y datos, utilizando las señales WR y RD para escribir y leer datos, respectivamente. Su capacidad de configurarse para trabajar en diferentes modos y con distintos tipos de señales lo hace ideal para manejar interfaces con dispositivos externos como displays, teclados, sensores, entre otros.

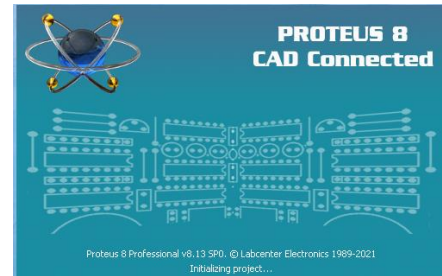
En el contexto de nuestro proyecto, el Intel 8255A será esencial para establecer la comunicación entre el microprocesador Intel 8086 y los displays de 14 segmentos. Utilizaremos los puertos A y B del PPI para enviar los datos que determinarán qué segmentos del display se deben activar, permitiendo así que se muestren caracteres o números específicos en el display. El puerto C se puede usar para manejar las señales de control que sincronizan la activación de los diferentes displays, asegurando que el sistema pueda multiplexar las letras o números en las distintas posiciones del display de manera eficiente.

Funcionamiento del circuito

Para lograr el objetivo de este proyecto, que consiste en mostrar un mensaje (como un nombre) en un display de 14 segmentos utilizando el microprocesador Intel 8086, utilizaremos una combinación de componentes clave y herramientas de simulación.

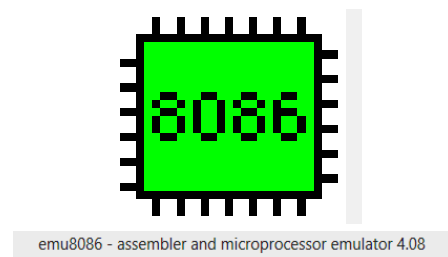
Software Proteus

Primero, emplearemos el software Proteus como entorno principal para el diseño y simulación del circuito, ya que permite integrar visualmente los componentes y comprobar su funcionamiento sin necesidad de montarlos físicamente.

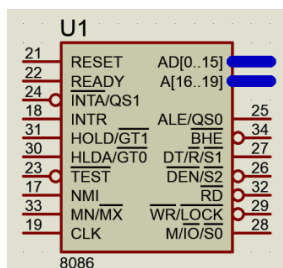


Software Emu8086

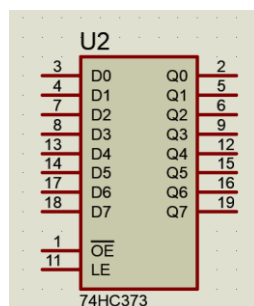
El programa en lenguaje ensamblador será desarrollado en el software Emu8086, que permite escribir, compilar y generar el archivo ejecutable (.exe) que contiene las instrucciones que controlarán el funcionamiento del sistema. Este archivo será cargado al microprocesador 8086, el cual es el encargado de ejecutar las instrucciones y coordinar la comunicación con los demás dispositivos.



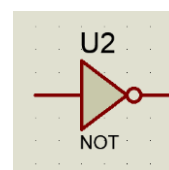
Los componentes que usaremos en circuit maker son:



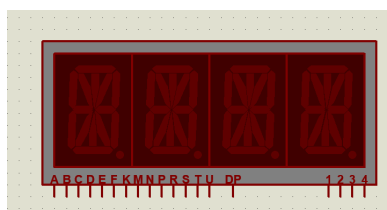
Microprocesador 8086



Latch 8 bits 74HC373



Puerta NOT
(Inversor)



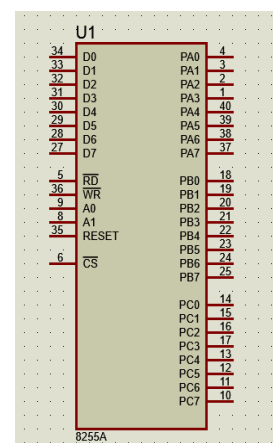
Display de 4 dígitos
(14SEG-MPX4-CC-RED)



Power
(VCC)

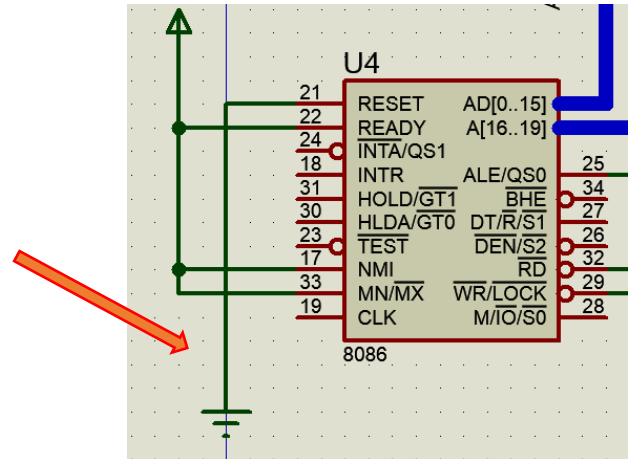


Ground (GND)



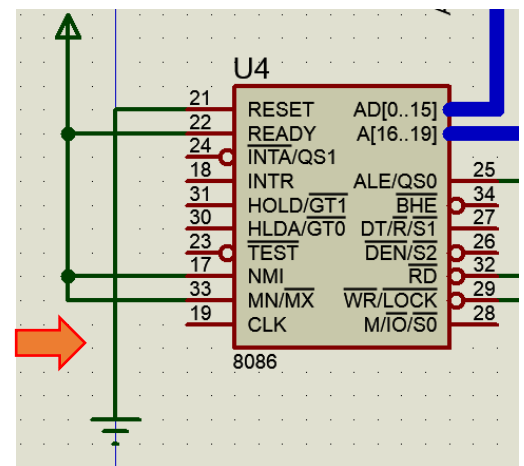
Intel 8255A (PPI –
Programmable Peripheral
Interface)

El pin RESET debe estar conectado a GND porque en este circuito no se necesita reiniciar el microprocesador manualmente. El pin RESET (reinicio) se utiliza para volver al estado inicial al microprocesador, es decir, para que empiece a ejecutar el programa desde el principio. Por otro lado, GND significa “tierra” y representa el nivel lógico bajo (0 voltios) en el circuito. Al conectar RESET a GND, se indica que no se quiere activar el reinicio. Esto es válido porque el programa en lenguaje ensamblador ya está cargado y se ejecuta automáticamente al iniciar la simulación en Proteus, sin necesidad de un pulso de arranque adicional.

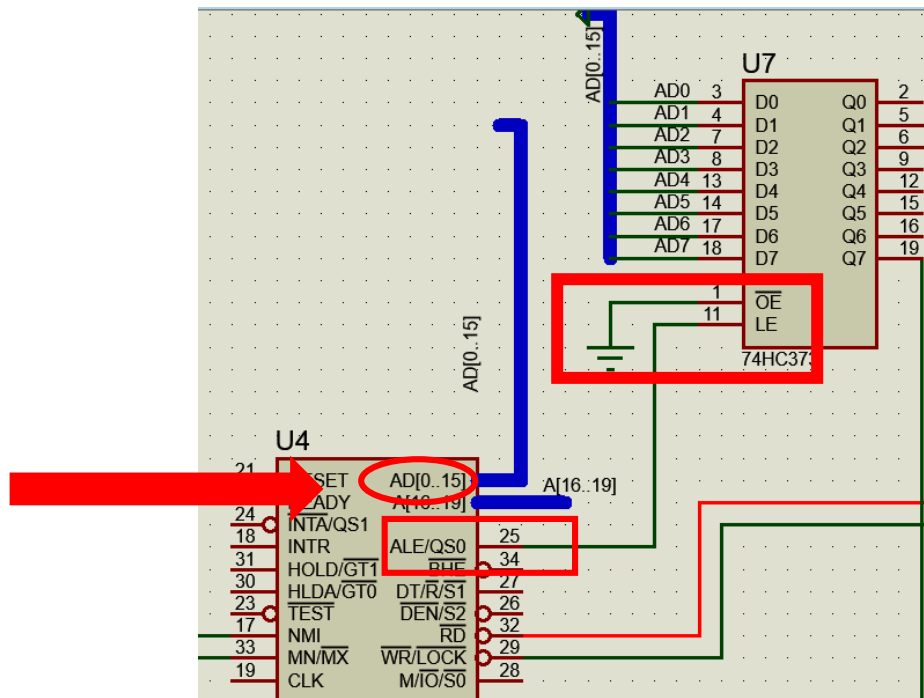


Los pines READY, NMI y $\overline{MN}/\overline{MX}$ están conectados al símbolo de Power (VCC) porque en este circuito se necesita que estos estén en estado alto (nivel lógico 1) para que el microprocesador funcione correctamente en modo simple. El símbolo **Power** representa la conexión a alimentación positiva del circuito, también llamada **VCC**, que generalmente es de 5 voltios.

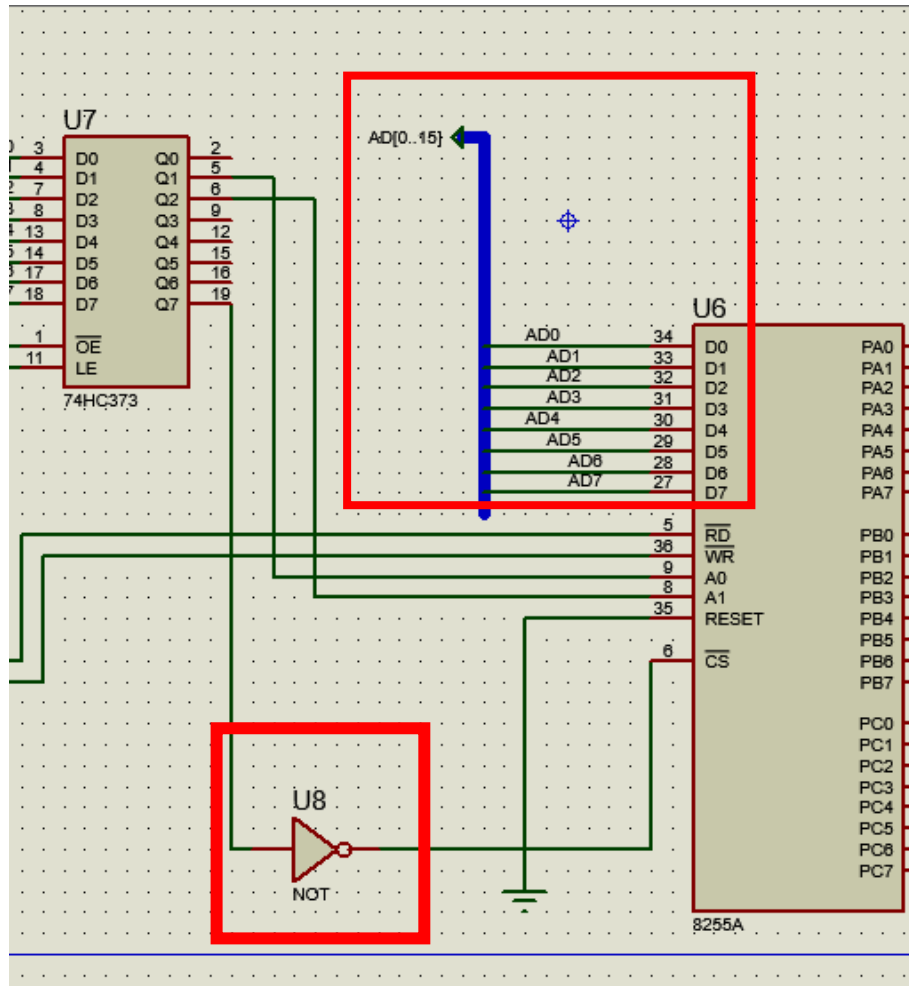
- **READY** es un pin que le indica al microprocesador si los dispositivos externos están listos para comunicarse. Al conectarlo a VCC, se le dice al 8086 que siempre puede continuar sin esperar, lo cual es ideal en simulaciones donde no hay retardos reales.
- **NMI** (Non-Maskable Interrupt) es una interrupción especial que se activa con un nivel bajo. Al conectarlo a VCC, se evita que se active accidentalmente, ya que en este proyecto no se usan interrupciones externas.
- **$\overline{MN}/\overline{MX}$** define el modo de operación del microprocesador: **modo mínimo** si está en alto, y **modo máximo** si está en bajo. Al conectarlo a VCC, el 8086 trabaja en **modo mínimo**, que es el más adecuado para un sistema sencillo como este, donde el microprocesador trabaja solo sin coprocesadores.



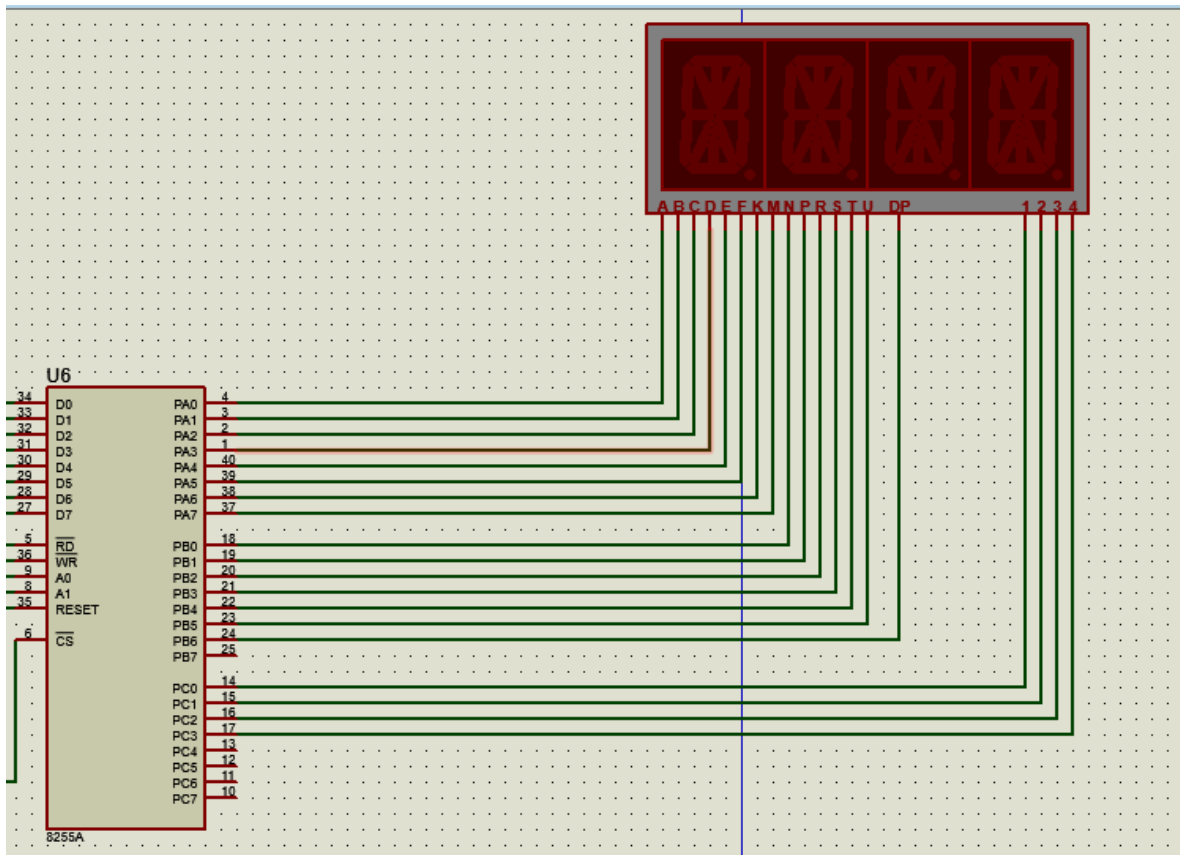
Para lograr que el microprocesador se comunique correctamente con el display, utilizaremos el circuito PPI 8255A, que actúa como interfaz programable de entrada/salida. Esta interfaz permite enviar datos desde el microprocesador hacia los puertos que controlan los segmentos del display. Sin embargo, dado que el bus AD[0..15] del 8086 es multiplexado (usa las mismas líneas para direcciones y datos), es necesario añadir un latch 74HC373, que se encargará de capturar y mantener la dirección durante el tiempo necesario, liberando luego el bus para el envío de datos.



El bus AD[0..15] del microprocesador Intel 8086 es un conjunto de 16 líneas que cumplen una doble función: transportar direcciones y datos, dependiendo del momento del ciclo de ejecución. Esta característica se conoce como bus multiplexado, y permite ahorrar pines físicos en el chip. En nuestro proyecto, este bus es fundamental, ya que a través de él el microprocesador se comunica con los periféricos, como el circuito 8255A, encargado de manejar el display. Sin embargo, como las líneas AD cambian su función durante la ejecución, necesitamos un componente que capture la dirección antes de que el bus comience a enviar datos. Para ello utilizamos un latch 74HC373, el cual guarda la dirección cuando el 8086 activa la señal ALE/QS0 (Address Latch Enable / Queue Status 0). En este caso, ALE se utiliza para indicar que en el bus AD[0..15] hay una dirección válida, por lo tanto se conecta directamente al pin LE (Latch Enable) del 74HC373. Esto permite que la dirección quede fijada en la salida del latch justo en el momento necesario, antes de que el bus cambie a transmitir datos. Este proceso es lo que técnicamente se conoce como separar las direcciones de los datos, ya que aunque físicamente comparten las mismas líneas, el latch permite diferenciarlos en el tiempo. Además, el pin OE (Output Enable) del latch se conecta a GND (nivel bajo) para que sus salidas estén siempre activas, es decir, que la dirección capturada esté disponible de forma constante para el resto del circuito. Finalmente, para mantener el diseño claro y ordenado en Proteus, usamos un bus lógico que conecta visualmente todas las líneas AD[0..15], evitando tener que hacer cada conexión por separado, lo que facilita la lectura y construcción del circuito.



En esta parte del circuito utilizamos el PPI 8255A, el cual actúa como la interfaz que conecta el microprocesador con los dispositivos externos, en este caso un display de 14 segmentos. El PPI se conecta al bus de datos AD[0..7], permitiendo el intercambio de datos entre el 8086 y los puertos del PPI (A, B y C). Para que el microprocesador pueda seleccionar a cuál puerto enviar datos, utilizamos las salidas Q1 y Q2 del latch 74HC373, que se conectan a los pines A0 y A1 del PPI, los cuales definen internamente qué puerto está siendo accedido. Además, la salida Q7 del latch se conecta, a través de una compuerta inversora (NOT), al pin CS (Chip Select) del PPI. Esto se hace porque el CS se activa con nivel bajo, y Q7 entrega nivel alto cuando se quiere habilitar el PPI; al invertir la señal, se logra que el PPI solo se active en el momento correcto, cuando ya se ha cargado la dirección. Este mecanismo evita que el PPI reciba direcciones en lugar de datos, y asegura que solo responda cuando realmente se quiere escribir o leer información. Finalmente, el pin RESET del 8255A se conecta a GND, ya que no se requiere reiniciarlo manualmente y, de este modo, el circuito entra directamente en funcionamiento desde el inicio de la simulación. Todo este conjunto permite que las instrucciones escritas en ensamblador accedan correctamente a los puertos del PPI mediante las instrucciones IN y OUT, y que los datos enviados por el microprocesador lleguen al display de forma controlada.



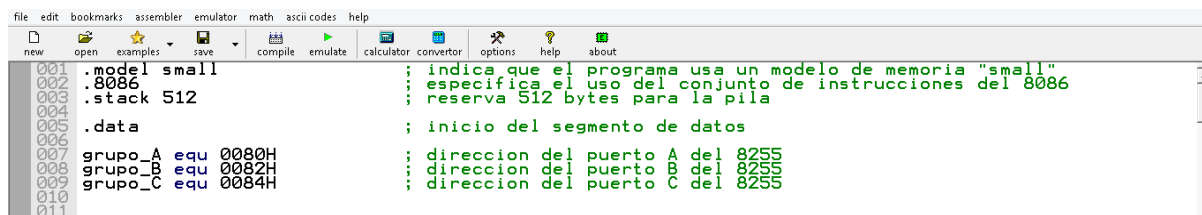
Finalmente, utilizamos un display de 14 segmentos de 4 dígitos para mostrar el mensaje programado. Este display está controlado a través del circuito PPI 8255A, el cual tiene tres puertos de entrada/salida de 8 bits: PA, PB y PC. En este diseño, se conectan los pines PA0 a PA7 a las líneas de control de los segmentos del display (por ejemplo, segmentos A, B, C, D, E, F, G, K, etc.), ya que cada bit puede encender o apagar un segmento específico según el carácter que se desea mostrar. Por otro lado, los pines del puerto PC0 a PC7 se utilizan para activar los dígitos del display, uno por uno, mediante la técnica de multiplexado. Así, aunque el display tiene varios dígitos, solo uno se activa a la vez, pero a una velocidad tan alta que el ojo humano los percibe todos encendidos simultáneamente. Esta configuración permite al microprocesador enviar un byte con el valor del carácter a mostrar por el puerto A, y activar el dígito correspondiente mediante el puerto C, repitiendo este proceso rápidamente para formar palabras o nombres completos en el display. Gracias al control flexible del 8255A y a su capacidad de manejar múltiples salidas en paralelo, es posible mostrar secuencias de texto de forma dinámica utilizando un solo microprocesador y una interfaz eficiente.

CÓDIGO DEL PROGRAMA

Para el correcto funcionamiento del programa diseñado en Emu8086 y su simulación en Proteus, es fundamental tener en cuenta los encabezados o directivas iniciales que establece el entorno del lenguaje ensamblador. Estos encabezados permiten definir el modelo de memoria, el tipo de microprocesador, la pila y las direcciones necesarias para la interacción con los periféricos del sistema.

Antes de desarrollar el cuerpo del programa, es indispensable definir:

- El modelo de memoria que se usará.
- La reserva de pila, necesaria para el manejo de subrutinas y almacenamiento temporal.
- Las direcciones de los puertos del PPI 8255, que permitirán el envío de datos al hardware externo (el display).



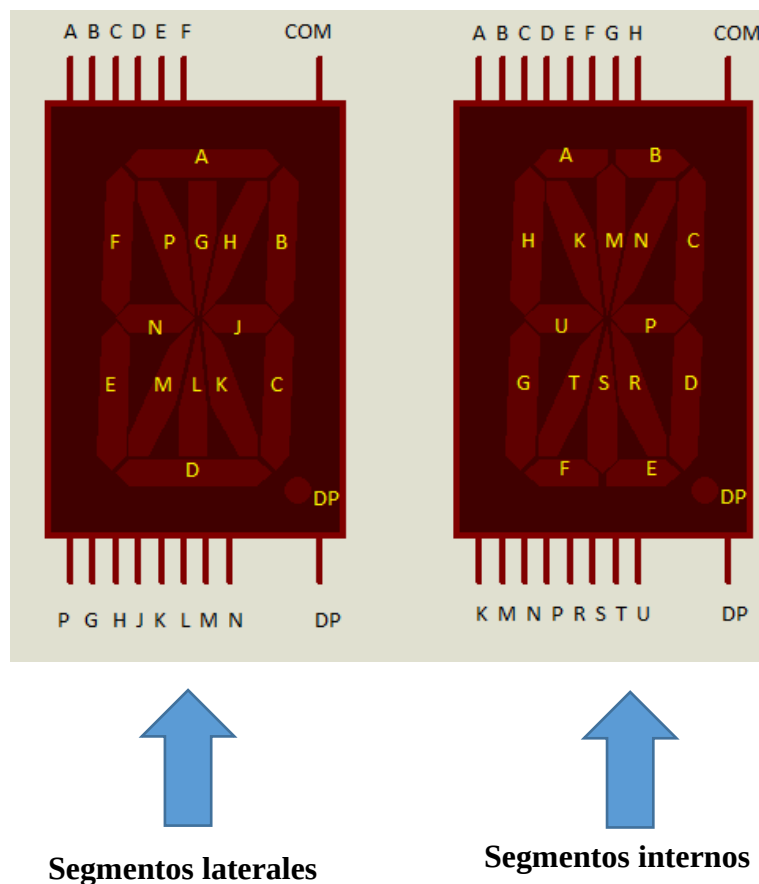
```
001 .model small          ; indica que el programa usa un modelo de memoria "small"
002 .8086                 ; especifica el uso del conjunto de instrucciones del 8086
003 .stack 512            ; reserva 512 bytes para la pila
004
005 .data                 ; inicio del segmento de datos
006
007 grupo_A equ 0080H      ; direccion del puerto A del 8255
008 grupo_B equ 0082H      ; direccion del puerto B del 8255
009 grupo_C equ 0084H      ; direccion del puerto C del 8255
010
011
```

El encabezado `.model small` indica que el programa utiliza un modelo de memoria del tipo "small", lo que significa que tanto el segmento de código como el de datos ocuparán un solo segmento de memoria de hasta 64 KB cada uno, lo cual es común en programas sencillos que no requieren una gestión compleja de memoria. Esto permite que las instrucciones y los datos estén organizados de manera compacta y eficiente. La directiva `.8086` especifica que el conjunto de instrucciones utilizado corresponde al microprocesador Intel 8086, lo cual es fundamental para que el ensamblador interprete correctamente las instrucciones y garantice la compatibilidad con el hardware simulado. Al definir explícitamente la arquitectura, se evita errores de ejecución y se facilita el uso adecuado de los registros y modos de direccionamiento disponibles en el 8086. Por su parte, la línea `.stack 512` reserva 512 bytes para la pila, una estructura de datos tipo LIFO (Last In, First Out) que almacena direcciones de retorno, variables temporales y otros datos durante la ejecución del programa, lo que es esencial para el uso seguro de instrucciones como `CALL`, `PUSH` y `POP`. La directiva `.data` marca el inicio del segmento de datos, donde se definen las constantes, variables y direcciones necesarias para la comunicación con los periféricos, separando claramente esta área de los datos del código y mejorando la organización del programa. Las líneas `grupo_A equ 0080H`, `grupo_B equ 0082H` y `grupo_C equ 0084H` definen las direcciones de los puertos A, B y C del **8255**, asignándoles nombres simbólicos para facilitar su uso posterior en el código. El puerto A y B se utilizan para enviar datos al display, mientras que el puerto C es comúnmente empleado para el control del sistema, como la generación de señales de habilitación y sincronización del hardware conectado. Esta organización permite un control más preciso y simplificado del comportamiento del sistema a través del programa.

Formación de Letras en el Display

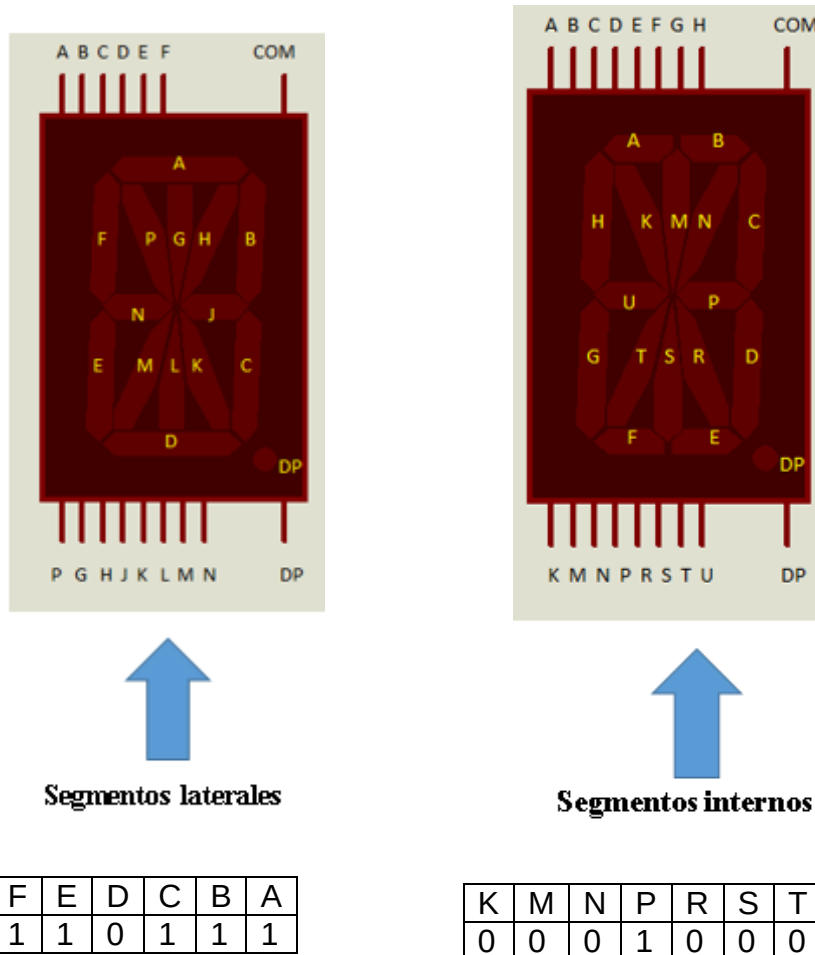
Para formar letras en el display de 14 segmentos, se hace uso de máscaras binarias o valores hexadecimales que representan el estado de encendido o apagado de cada uno de los segmentos individuales del display. Cada valor define una configuración específica que activa ciertos segmentos para representar visualmente una letra o símbolo determinado.

Estas configuraciones se dividen en dos partes: una correspondiente a los segmentos laterales (por ejemplo, los que forman los bordes verticales y horizontales exteriores), y otra correspondiente a los segmentos internos (como diagonales o cruces centrales). En el código, estas combinaciones se definen mediante constantes simbólicas (por ejemplo, A1, A2, B1, B2, etc.), donde cada par forma la representación completa de una letra al controlar dos registros o salidas diferentes del sistema.



Ejemplo: Representación de la letra “A” en un display de 14 segmentos

Para representar correctamente la letra A en un display de 14 segmentos, es necesario encender tanto los segmentos laterales como los segmentos internos que definen su forma característica.



→ 37H Convertimos a valores hexadecimales. → 22H

Una vez convertidos los valores binarios en su equivalente hexadecimal, se utilizan dentro del programa ensamblador mediante la directiva equ, que permite asignar un nombre simbólico a cada valor. Para representar correctamente una letra en el display de 14 segmentos, se definen dos variables (por ejemplo, A1 y A2) que almacenan los valores hexadecimales correspondientes a los segmentos internos y laterales del carácter. Estas variables facilitan la escritura y comprensión del código, ya que permiten reutilizar los patrones de activación para cada letra o símbolo sin necesidad de escribir los valores directamente en cada instrucción de salida hacia los puertos del PPI 8255.

```

010
011
012
013 ;Letras y simbolos
014 A1 equ 22H
015 A2 equ 37H
016

```

De esta forma, es posible formar tanto letras como números en el display de 14 segmentos, asignando para cada uno de ellos un conjunto específico de segmentos a encender. Mediante el uso de valores hexadecimales representados con nombres simbólicos (equ), el programa puede controlar con precisión qué partes del display se activan, permitiendo visualizar de manera clara los caracteres deseados. Esta técnica facilita la organización del código y hace más eficiente la construcción de mensajes personalizados que combinen letras y cifras.

```

010
011
012
013 ;Letras y simbolos personalizados para el display
014 A1 equ 22H
015 A2 equ 37H
016
017 I
018 C1 equ 00H
019 C2 equ 39H
020
021 E1 equ 22H
022 E2 equ 39H
023
024
025 I1 equ 08H
026 I2 equ 89H
027
028 J1 equ 00H
029 J2 equ 1EH
030
031
032 L1 equ 00H
033 L2 equ 38H
034
035
036 01 equ 00H
037 02 equ 3FH
038
039
040
041 V1 equ 11H
042 V2 equ 30H
043
044 111 equ 00h
045 112 equ 36H
046

```

```

047 ;Numeros (xx = segmentos, yy = leds de fondo)
048 yy0 equ 00H
049 xx0 equ 3FH
050 yy1 equ 00H
051 xx1 equ 06H
052 yy2 equ 22H
053 xx2 equ 5BH
054 yy3 equ 22H
055 xx3 equ 0FH
056 yy4 equ 22H
057 xx4 equ 66H
058 yy5 equ 22H
059 xx5 equ 2DH
060 yy6 equ 22H
061 xx6 equ 3DH
062 yy7 equ 00H
063 xx7 equ 07H
064 yy8 equ 22H
065 xx8 equ 3FH
066 yy9 equ 22H
067 xx9 equ 27H
068 null equ 00H

```

Macro para el Envío de Datos al Display de 14 Segmentos

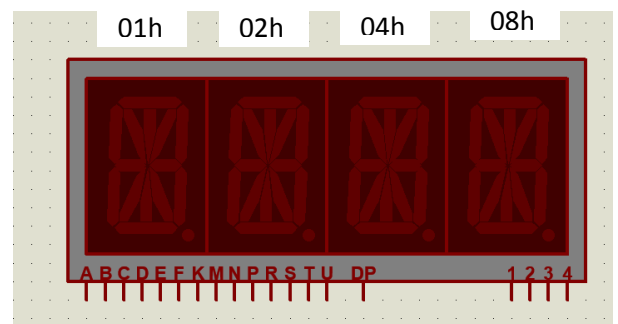
```
071  
072 print macro dato1,dato2,selec  
073 mov al,dato2  
074 MOV DX,GRUPO_A  
075 OUT DX,AX  
076 mov al,dato1  
077 MOV DX,GRUPO_B  
078 OUT DX,AX  
079 mov al,selec  
080 not ax  
081 MOV DX,GRUPO_C  
082 out DX,ax  
083 mov al,00h  
084 not ax  
085 MOV DX,GRUPO_C  
086 out DX,ax  
087 endm
```

En este fragmento de código se define una macro instrucción llamada `print` que recibe tres parámetros (`dato1`, `dato2` y `selec`) con el objetivo de simplificar la escritura del programa, ya que las instrucciones se repiten para cada carácter a mostrar en el display. Primero, se carga `dato2` en el registro `AL` y se envía al Grupo A (`GRUPO_A`) del PPI 8255, que controla una parte de los segmentos del display. Luego se carga `dato1` y se envía por el Grupo B (`GRUPO_B`), correspondiente a la otra parte de los segmentos. A continuación, se carga el valor de `selec`, que indica cuál dígito del display debe encenderse, y se invierte (`NOT AX`) ya que el display es de cátodo común, lo que implica lógica activa baja; este valor invertido se envía al Grupo C (`GRUPO_C`) para activar el dígito correspondiente. Finalmente, se desactiva la posición encendida enviando `00H` al grupo C (también invertido) para evitar que el dígito permanezca encendido al mostrar el siguiente carácter. Esta macro permite reutilizar la secuencia para mostrar múltiples letras o números de manera ordenada y eficiente.

Control de Múltiples Dígitos en Display de 14 Segmentos mediante Macro `print`

En este bloque del programa, se utiliza correctamente la macro `print` para mostrar secuencialmente las letras del nombre "JOEL" en un display de 14 segmentos de 4 dígitos. Cada llamada a la macro recibe tres parámetros: dos valores (`dato1` y `dato2`) que definen los segmentos a encender para formar una letra específica, y un tercer valor (`selec`) que indica cuál de los cuatro displays debe activarse. Cada display tiene un latch o línea de habilitación única, activada mediante una máscara de selección (`01h`, `02h`, `04h`, `08h`), que permite controlar individualmente cada dígito del display. Por ejemplo, `01h` habilita el primer display (posición 1), `02h` el segundo, y así sucesivamente hasta `08h` para el cuarto. Al iterar con un bucle controlado por el registro `CX`, el programa repite la secuencia 1000 veces, asegurando que las letras permanezcan visibles en sus respectivas posiciones el tiempo suficiente para ser leídas correctamente. Esta técnica de multiplexado controlado por software permite representar distintos caracteres en cada dígito del display usando una sola

```
120 mov cx, 1000  
121 mostrarjoel4:  
122 print J1, J2, 01h  
123 print 01, 02, 02h  
124 print E1, E2, 04h  
125 print L1, L2, 08h  
126 loop mostrarjoel4
```



```

372 .code
373 jmp inicio
374
375 inicio:
376 mov ax, @data
377 mov ds, ax
378
379 ; Configurar el 8255
380 mov al, 80h
381 mov dx, grupo_C + 2
382 out dx, al
383
384 jmp $
385
386 end inicio

```

Finalmente, en esta sección del programa se realiza la configuración inicial del circuito integrado 8255, necesario para establecer el modo de operación de sus puertos. Primero, el código salta a la etiqueta inicio mediante la instrucción `jmp inicio`. Luego, se inicializa el segmento de datos cargando la dirección de `@data` en el registro AX y transfiriéndola al registro DS. Posteriormente, se prepara el valor 80h en el registro AL, que corresponde a una configuración del 8255 en modo 0, con todos los puertos (A, B y C) establecidos como salidas. Este valor se envía al registro de control del 8255, ubicado en la dirección `grupo_C + 2` (la dirección de control del chip). La instrucción `out dx, al` envía este byte de configuración para activar el modo deseado. Finalmente, el programa entra en un bucle infinito con `jmp $`, lo que mantiene el sistema en espera, ya sea para mantener los valores actuales o para que otros procesos intervengan si se extiende el código.

Simulación de Desplazamiento de Texto en Display de 14 Segmentos

Una vez comprendido el funcionamiento del display y cómo se muestran las letras de forma estática mediante la macro `print`, es posible simular un efecto de desplazamiento o "scroll" haciendo uso de ciclos controlados por tiempo. Este desplazamiento se logra activando y desactivando los distintos dígitos del display de manera secuencial, moviendo progresivamente cada letra de izquierda a derecha (o viceversa). Para lograrlo, se debe encender una letra en un dígito, luego apagarla y encenderla en el siguiente, combinando dos o más letras por momentos para crear la ilusión de movimiento continuo. Esto se controla utilizando bucles (loop) y retardos temporales (delays), permitiendo que cada combinación de letras se mantenga visible por un breve intervalo antes de pasar a la siguiente. Al repetir este proceso con distintas posiciones y combinaciones, se obtiene un efecto visual similar al desplazamiento de texto, muy útil para mostrar palabras o mensajes más largos en un display con número limitado de dígitos.

Ejemplo de la Simulación: Desplazamiento del Nombre “JOEL”

